Exercise 1

AIM :

To write a simple code to get two variable one as integer and another as float and display the given value as output also print the size of both the variables. And also to perform all stages of compilation process one by one (preprocessing, Compiling, assembling, linking).

ALGORITHM :

Step 1: START

Step 2: Include the standard input-output header file (stdio.h)

Step 3: Declare main() function

Step 4: Declare variables:

- intVar (integer data type)
- floatVar (floating-point data type)

Step 5: Display prompt message: "Enter an integer value: "

Step 6: Read integer input from user using scanf() with %d format specifier

Step 7: Store input value in intVar

Step 8: Display prompt message: "Enter a float value: "

Step 9: Read float input from user using scanf() with %f format specifier

Step 10: Store input value in floatVar

Step 11: Display section header: "--- Variable Details ---"

Step 12: Display integer value using printf() with %d format specifier

Step 13: Display float value with 2 decimal precision using printf() with %.2f

Step 14: Display size of integer variable using sizeof() operator

Step 15: Display size of float variable using sizeof() operator

Step 16: Return 0 indicating successful program execution

Step 17: STOP

PROGRAM :

```
#include <stdio.h>
int main() {
```

```
    // Declare variables
    int intVar;
    float floatVar;
    // Input values
    printf("Enter an integer value: ");
    scanf("%d", &intVar);
    printf("Enter a float value: ");
    scanf("%f", &floatVar);
    // Display values
    printf("\n--- Variable Details ---\n");
    printf("Integer value: %d\n", intVar);
    printf("Float value: %.2f\n", floatVar);
    // Display sizes
    printf("Size of integer variable: %zu bytes\n", sizeof(intVar));
    printf("Size of float variable: %zu bytes\n", sizeof(floatVar));
    return 0;
}
```

COMPILATION PROCESS :

Phase I: PREPROCESSING

Input: EX1.c (Source Code)

Process:

1. Remove all comments
2. Expand #include directives
3. Process macro definitions
4. Generate intermediate file with .i extension

Output: EX1.i (Preprocessed Code)

Phase II: COMPILATION

Input: EX1.i (Preprocessed Code)

Process:

1. Syntax and semantic analysis

2. Convert C code to assembly language

3. Code optimization

4. Generate assembly code file

Output: EX1.s (Assembly Code)

Phase III: ASSEMBLY

Input: EX1.s (Assembly Code)

Process:

1. Convert assembly mnemonics to machine code

2. Generate relocatable object code

3. Create symbol table

Output: EX1.o (Object File)

Phase IV: LINKING

Input: EX1.o (Object File)

Process:

1. Resolve external references

2. Link with standard C library (libc)

3. Combine object files

4. Generate executable file

Output: EX1.exe (Executable Program)

Phase V: EXECUTION

Input: EX1.exe (Executable)

Process:

1. Load program into memory

2. Execute instructions sequentially

3. Display output

4. Terminate program

Output: Program results on console

OUTPUT :

```
D:\C programming>EX1.exe
Enter an integer value: 55
Enter a float value: 55

--- Variable Details ---
Integer value: 55
Float value: 55.00
Size of integer variable: 4 bytes
Size of float variable: 4 bytes
```